

NOIP 2026 模拟赛

时间：2026 年 7 月 13 日

题目名称	桥牌	游戏	涂色	括号
题目类型	传统题	传统题	传统题（SPJ）	传统题
目录	bridge	game	color	brackets
可执行文件名	bridge	game	color	brackets
输入文件名	bridge.in	game.in	color.in	brackets.in
输出文件名	bridge.out	game.out	color.out	brackets.out
每个测试点时限	1 秒	1 秒	1 秒	2.5 秒
内存限制	512 MB	512 MB	512 MB	512 MB
子任务数目	10	20	25	25
测试点是否等分	是	是	是	是

提交源程序文件名

对于 C++ 语言	bridge.cpp	game.cpp	color.cpp	brackets.cpp
-----------	------------	----------	-----------	--------------

编译选项

对于 C++ 语言	-O2 -std=c++14
-----------	----------------

注意事项（请仔细阅读）

1. 测试机器：CPU(AMD Ryzen 7 5700G 3.80 GHz), RAM 32G。
2. 测试环境：Windows 10, g++ 13.2.0, Lemonlime 0.3.4。
3. 文件名（程序名和输入输出文件名）必须使用英文小写。
4. C/C++ 中函数 `main()` 返回类型必须是 `int`，程序正常结束返回值必须是 0。
5. 若无特殊说明，结果的比较方式为全文比较（过滤行末空格及文末回车）。
6. 选手提交的程序源文件必须不大于 100KB。
7. 程序可使用的栈空间内存限制与题目的内存限制一致。

桥牌 (bridge)

【题目描述】

简单来说, 桥牌共有四个人, 顺次按照北、东、南、西的顺序顺次坐在一个方形桌子的四边, 其中南北一队, 东西一队。在桥牌中, 除去大小王共 52 张牌, 分给每人 13 张, 经过叫牌、打牌两个阶段。

其中有一个非常重要的概念, 叫做“大牌点”(High Card Points), 具体计算方式如下:

- 手牌中每有一个 A, 加 4 点。
- 手牌中每有一个 K, 加 3 点。
- 手牌中每有一个 Q, 加 2 点。
- 手牌中每有一个 J, 加 1 点。

显然所有人的大牌点加起来是 40, 而每个人会拥有不多于 37 个大牌点。

现在, 你知晓了你自己和明手(另一人)的手牌, 请你求出另外两人一人手中最多和最少有多少个大牌点。

【输入格式】

从文件 *bridge.in* 中读入数据。

第一行一个正整数 T , 表示测试数据组数。

接下来对于每组数据, 输入 8 行, 每行一个由字母和数字组成的字符串, 分别表示你的黑桃、红桃、方片、草花和明手的黑桃、红桃、方片、草花, 其中 10 用 T 表示, 保证每人都恰有 13 张牌、牌没有重复。

请注意, 如果一个人没有某一门花色, 那么在对应的输入中将输入一个空行。

【输出格式】

输出到文件 *bridge.out* 中。

输出两个数, 表示另两人中一人手中最少和最多有多少大牌点。

【样例 1 输入】

```
1 2
2 T9874
3 K97642
4 K3
5
6 A632
7
8 T9852
9 KQJ6
10 T98765432
11 T987
12
13
14 T98765432
15 T987
16
17
```

【样例 1 输出】

```
1 0 24
2 3 37
```

【样例 2】

见选手目录下的 *bridge/bridge2.in* 与 *bridge/bridge2.ans*。

【数据范围】

对于所有数据，满足 $T \leq 1000$ ，保证数据合法。

测试点编号	特殊限制
1 ~ 3	保证数据随机生成
4 ~ 5	保证你的手牌中至少有 10 点
6 ~ 10	无

游戏 (game)

【题目描述】

共有 n 张两面的卡牌，按照 $1 \sim n$ 的顺序排列。卡牌一面黑一面白，初始情况下一些卡牌黑面向上（用 0 表示），剩下的白面向上（用 1 表示）。

小 C 和小 D 轮流操作，小 C 先进行操作。每次每个人必须选择连续的 k 张牌，将这些牌全部翻转为黑面朝上或全部翻转为白面朝上。例如，当 $n = 5, k = 3$ 时，假如目前卡牌的状态是 **01010**，那么一个人可以将卡牌状态变为 **11110**、**00010**、**01110** 等，但不能变成 **10110** 或 **01010**。

如果一次操作之后，所有卡牌都变成黑面朝上或都变成白面朝上，那么进行这个操作的人获胜。

小 C 和小 D 都绝顶聪明，他们都使用最优策略，即尽量让自己获胜、如无法获胜则尽量不让对方获胜。

请你判断游戏的结果。

【输入格式】

从文件 *game.in* 中读入数据。

第一行一个正整数 T ，表示测试数据组数。

接下来对于每组数据，第一行两个正整数 n, k ，第二行一个长度为 n 的 01 串，表示初始卡牌状态。

【输出格式】

输出到文件 *game.out* 中。

输出 T 行，每组数据输出一行。对于每组数据，如果小 C 会赢，输出 **win**；如果小 D 会赢，输出 **lose**；否则输出 **tie**。

【样例 1 输入】

```
1 3
2 4 2
3 0101
4 4 1
5 0101
6 4 2
```

7 0011

【样例 1 输出】

1 lose
2 tie
3 win

【样例 2】

见选手目录下的 *game/game2.in* 与 *game/game2.ans*。
该样例满足测试点 1 ~ 5 的限制。

【样例 3】

见选手目录下的 *game/game3.in* 与 *game/game3.ans*。
该样例满足测试点 9 ~ 13 的限制。

【数据范围】

对于所有数据，满足 $1 \leq T \leq 40$ ， $1 \leq k \leq n \leq 10^5$ 。

测试点编号	n	k	特殊限制
1 ~ 5	≤ 10	≤ 10	无
6 ~ 8	$\leq 10^3$	≤ 2	无
9 ~ 13	$\leq 10^3$	$\leq 10^3$	无
14 ~ 16	$\leq 10^5$	$\leq 10^5$	保证小 D 不会获胜
17 ~ 20	$\leq 10^5$	$\leq 10^5$	无

涂色 (color)

【题目描述】

小 C 有一个 $n \times m$ 的网格，他可以给每个格子涂上黑色或白色当中的一个。另外他还有 k 个要求，每个要求形如“两个相邻格子颜色相同/不相同”，两个格子 (x_1, y_1) 和 (x_2, y_2) 相邻当且仅当

$$|x_1 - x_2| + |y_1 - y_2| = 1,$$

保证对一对格子的限制至多只有一个。

当然，由于这些限制都是由小 C 的直觉给出的，所以可能不能同时满足，因此，他允许你违反其中至多 $k_0 \geq \frac{k}{4}$ 个限制，请你构造任意一组方案，数据保证有解。

【输入格式】

从文件 `color.in` 中读入数据。
第一行一个正整数 T ，表示测试数据组数。
对于每组数据，第一行四个非负整数 n, m, k, k_0 。
接下来 k 行，每行四个非负整数 x_1, y_1, x_2, y_2, op ，描述一条 (x_1, y_1) 和 (x_2, y_2) 之间的限制， $op = 0$ 表示限制两个格子相同， $op = 1$ 表示限制两个格子不相同。

【输出格式】

输出到文件 `color.out` 中。
对于每组数据，输出 n 行，每行一个长度为 m 的 01 字符串。第 i 行第 j 个字符为 0 表示格子 (i, j) 涂白，为 1 表示格子 (i, j) 涂黑。

【样例 1 输入】

```
1 4
2 4 4 2 2
3 4 4 3 4 0
4 2 3 1 3 0
5 2 3 6 3
6 1 1 2 1 0
7 1 3 2 3 1
8 1 2 1 1 0
```

```
9  2 2 1 2 0
10 2 3 2 4 1
11 1 2 1 3 1
12 4 4 6 3
13 3 3 3 2 1
14 2 4 3 4 1
15 4 2 4 1 1
16 3 4 3 5 1
17 1 3 1 4 0
18 2 3 2 4 1
19 4 4 6 3
20 1 3 2 3 1
21 4 4 3 4 1
22 1 1 2 1 1
23 3 2 3 3 0
24 4 3 3 3 0
25 2 2 1 2 0
```

【样例 1 输出】

```
1  0000
2  0010
3  0010
4  0011
5  001
6  000
7  0000
8  0001
9  1100
10 1000
11 0000
12 1110
13 1110
14 1101
```

【样例 2】

见选手目录下的 `color/color2.in` 与 `color/color2.ans`。
该样例满足测试点 5 ~ 6 的限制。

【样例 3】

见选手目录下的 `color/color3.in` 与 `color/color3.ans`。
该样例满足测试点 16 ~ 18 的限制。

【数据范围】

对于所有数据，满足

$$T \leq 200, \quad 1 \leq n, m \leq 10^3, \quad \sum n, \sum m \leq 5 \times 10^3,$$
$$0 \leq k \leq n(m - 1) + (n - 1)m, \quad \sum k \leq 2 \times 10^6, \quad \frac{k}{4} \leq k_0 \leq k.$$

测试点编号	n	m	k	k_0
1 ~ 4	≤ 4	≤ 4	≤ 24	$\geq \frac{k}{4}$
5 ~ 6	≤ 50	≤ 50	≤ 10	$\geq \frac{k}{4}$
7 ~ 8	≤ 50	≤ 50	≤ 20	$\geq \frac{k}{4}$
9 ~ 10	$\leq 10^3$	$\leq 10^3$	≤ 10	$\geq \frac{k}{4}$
11 ~ 13	$\leq 10^3$	$\leq 10^3$	≤ 20	$\geq \frac{k}{4}$
14 ~ 15	$= 1$	$\leq 10^3$	$\leq 2 \times 10^6$	$\geq \frac{k}{4}$
16 ~ 18	$\leq 10^3$	$\leq 10^3$	$\leq 2 \times 10^6$	$\geq \frac{k}{2}$
19 ~ 20	$\leq 10^3$	$\leq 10^3$	$\leq 2 \times 10^6$	$\geq \frac{k}{3}$
21 ~ 25	$\leq 10^3$	$\leq 10^3$	$\leq 2 \times 10^6$	$\geq \frac{k}{4}$

【提示】

本题下发了特殊校验器 `color/grader.cpp` 供本地调试使用，你不需要了解特殊校验器的工作原理，最终使用的校验器可能与下发的有所区别。

首先，确保 `color` 目录下包含 `testlib.h` 和 `grader.cpp` 两个文件，随后，打开终端并进入 `color` 目录下，输入 `g++ -o grader grader.cpp -O2` 编译校验器，随后目录下会出现名为 `grader` 的可执行文件。

在终端中校验器将接受三个参数，用法为 `./grader <in> <out> <ans>`。

例如若你想要测试第一个样例，你的程序输出为 `color.out`，则你在终端中输入 `./grader color1.in color.out color1.ans`。

如你的输出合法，则校验器将返回 0 并输出 `ok`。

如你的输出格式不合法或构造错误，则校验器会返回非 0 值并输出 `wrong answer`。

上述方法请在 Linux 系统下使用，推荐使用 NOI Linux 进行测试。

在 Windows 下，同样地，将 `testlib.h`、`grader.cpp` 和输入文件、你的输出文件，以及标准输出文件放在同一个文件夹下，编译 `grader.cpp`，然后用 `cmd` 打开该文件夹。

运行命令 `./grader color1.in color.out color1.ans` 即可，如果你需要测试其他样例，对应地将命令中的文件名修改为其他样例的文件名即可。

括号 (brackets)

【题目描述】

定义一个只包含 (和) 的字符串 s 为括号串, 并定义合法括号串如下:

- 空串是合法括号串。
- 如果 A 和 B 都是合法括号串, 那么 AB 也是合法括号串。
- 如果 A 是合法括号串, 那么 (A) 也是合法括号串。

现在小 C 有一个长度为 $2n$ 的括号串 s , 初始不一定合法, 他可以花费一些代价修改一些位置, 使得整个括号串合法。具体地, 如果他修改了 s_i , 那么他需要花费 a_i 的代价, 如果他不需修改任何位置, 代价为 0。

小 C 想请你帮忙计算他花费的最小代价, 同时, 他另有 q 次修改, 每次永久地修改一个 a_i , 请你分别求出初始和进行完每次修改后的答案。

【输入格式】

从文件 **brackets.in** 中读入数据。

第一行两个正整数 n, q 。

接下来一行 $2n$ 个非负整数 $a_1, \dots, a_{2n}$ 。

接下来一行一个长为 $2n$ 的括号串 s 。

接下来 q 行, 每行两个非负整数 i, x , 表示将 a_i 修改为 x 。

【输出格式】

输出到文件 **brackets.out** 中。

输出 $q + 1$ 行, 顺序表示初始和进行完每次修改后的答案。

【样例 1 输入】

```
1 3 5
2 9 2 2 2 2 10
3 () ) (( (
4 3 7
5 5 10
6 6 5
7 2 2
8 6 2
```

【样例 1 输出】

```
1 14
2 14
3 14
4 9
5 9
6 6
```

【样例 2 输入】

```
1 5 10
2 5 10 6 8 3 6 2 1 16 11
3 (((()))(((((
4 4 9
5 9 9
6 2 11
7 4 20
8 7 9
9 1 5
10 9 16
11 8 4
12 2 18
13 4 20
```

【样例 2 输出】

```
1 12
2 12
3 12
4 12
5 12
6 12
7 12
8 12
9 15
```

10 15

11 15

【样例 3】

见选手目录下的 `brackets/brackets3.in` 与 `brackets/brackets3.ans`。
该样例满足测试点 1 ~ 4 的限制。

【样例 4】

见选手目录下的 `brackets/brackets4.in` 与 `brackets/brackets4.ans`。
该样例满足测试点 9 ~ 11 的限制。

【样例 5】

见选手目录下的 `brackets/brackets5.in` 与 `brackets/brackets5.ans`。
该样例满足测试点 15 ~ 17 的限制。

【数据范围】

对于所有数据，满足 $1 \leq n, q \leq 10^5$ ， $0 \leq a_i, x \leq 10^9$ ， s 为括号串。

测试点编号	n	q	特殊限制
1 ~ 4	≤ 5	≤ 100	无
5 ~ 6	≤ 13	≤ 100	无
7 ~ 8	≤ 100	≤ 100	无
9 ~ 11	$\leq 10^3$	$\leq 2 \times 10^3$	A
12 ~ 14	$\leq 10^5$	≤ 10	A
15 ~ 17	$\leq 10^5$	$\leq 10^5$	A
18	$\leq 10^5$	$\leq 10^5$	B
19 ~ 21	$\leq 10^3$	$\leq 2 \times 10^3$	无
22 ~ 25	$\leq 10^5$	$\leq 10^5$	无

特殊限制 A：保证 s 在所有括号串中等概率随机生成。
特殊限制 B：保证 $a_i = x = 1$ 。